TECHNICAL REPORT

Interactive Batch ORAM with Garbled Circuits and Secret-Shared Shuffling

Scheme, security conditions, and reproducible resource estimates

16 October 2026 · Two-party semi-honest setting

Abstract

We study a private key–value service combining two-choice cuckoo hashing, recursive Circuit-OPRAM-style batching, streamed garbled circuits, and an interactive secret-shared shuffle. Both the database and each 256-bit query key are private; neither server is assumed to know the query. An AES-CMAC PRF computes both cuckoo indices inside the secure computation. Our main optimization matches requests to pool records, attaches secret request tags, shuffles the tagged records with a permutation unknown to either party, and opens only the shuffled tags. Payload extraction and removal then become local operations on shares. A fixed number of uniquely tagged dummies hides pool misses and duplicate requests.

We derive a complete accounting ledger for the selected compilation template, including garbled tables, both directions of OT traffic, shuffle preprocessing, conversion between representations, recursive-map updates, persistent state, and modeled storage I/O. The primary estimate uses ordinary Boolean records outside the shuffle; it does not require an unverified two-label conditional swap. Packed-value storage is reported as a conditional sensitivity. At 10,000 lookups/s, the selected batch has 32 requests for both studied sizes. The resulting historical-price network, state-capacity, and modeled-I/O budgets are \$39.05M and \$51.40M per year. These are capacity scenarios, not measured deployment costs or throughput guarantees: the inherited parallel eviction and grouping routines remain circuit composition allowances, and CPU, state service time, initialization, and overflow probabilities require validation.

Primary selected scenario	$N = 2^{28}$	$N = 2^{32}$
Batch / map fanout	32 / 4	32 / 4
Ordinary ANDs/query (million)	6.997	8.867
Total framed traffic/query (MB)	237.832	301.293
NIC hosts per party	298	378
Persistent state per party (TB)	64.70	1,061.58
Network + capacity + modeled I/O/year	\$39.05M	\$51.40M

Interpretation. The report provides a concrete pool-fetch protocol and a reproducible cost model for the surrounding construction. It does not claim that the whole construction has been implemented or that the chosen capacity parameters have a service-lifetime failure bound.

Contents

1 Functionality, threat model, and notation **3**

2 Dictionary and recursive ORAM layout **3**

3 Cryptographic primitives and representation costs **4**
3.1 Hashing a secret-shared 32-byte query key . . . 4
3.2 Packed storage and conditional movement are separate costs . . . 5
3.3 One-hot garbling: what is obtained and what is charged . . . 5
3.4 One-AND swaps and a fused flat-array exchange . . . 6

4 Interactive pool retrieval: protocol and security conditions **6**
4.1 Interface and invariants . . . 6
4.2 Match, tag, shuffle, and extract . . . 7
4.3 Composition with recursion, paths, and updates . . . 8
4.4 Security argument and its scope . . . 8
4.5 Representations and explicit communication charges . . . 9

5 Full batch execution and state dependencies **10**

6 Deriving the circuit and communication ledger **11**
6.1 Reusable Boolean kernels . . . 11
6.2 Work retained at each recursive tree . . . 11
6.3 Recursive updates, flat map, and application logic . . . 12
6.4 The new pool term and directional traffic . . . 12

7 Numerical results and selection of parameters **13**
7.1 Batch and recursion search . . . 13
7.2 Instantiated recursive trees . . . 13
7.3 Ordinary garbled-circuit work . . . 14
7.4 Pool protocol: all costs, not only online bytes . . . 14
7.5 Complete directional communication . . . 15
7.6 Packed-value sensitivity without an unsupported gate discount . . . 15

8 Persistent state, I/O, and annual capacity cost **15**
8.1 Historical deployment assumptions . . . 16
8.2 Selected annual budgets . . . 17
8.3 Known-work CPU sensitivity . . . 17

9 Throughput, interaction, and unresolved costs **17**

10 Reproduction and implementation checkpoints **18**

1 Functionality, threat model, and notation

Two noncolluding servers, a garbler G and evaluator E, hold a private key–value dictionary. We assume a static, semi-honest adversary corrupting at most one server. A query key $x \in \{0,1\}^{256}$ is supplied as $x = x_G \oplus x_E$; neither share alone reveals x. Database initialization must similarly establish a secret shared or encoded dictionary. Both cuckoo probes execute on every lookup. The returned value and hit bit remain shared between the servers; an authorized client may reconstruct them through a separately specified delivery interface. No server receives the cleartext query, cuckoo index, or value by default.

This report models application *reads*. Each read still changes the ORAM position maps, locations, validity bits, and stash/pool state. Arbitrary application writes, concurrent database mutation, client authentication, malicious security, availability, and durable recovery need additional protocols. Public workload information includes the fixed batch size, provisioned capacities, execution schedule, and completed-query count.

Symbol	Meaning
$N = 2^d$	Number of dictionary records; $d \in \{28, 32\}$
Q	Aggregate rate, 10,000 complete dictionary lookups/s
B	Fixed batch size; both tables receive B requests
h_ℓ	Logical-address/leaf width at recursive tree ℓ
$F = 2^f$	Number of child position labels packed in one map record
Z	Tree bucket capacity, fixed at 3 in this study
σ	Additional pool slack, fixed at 128 records
$r = \log_2(2B)$	Number of upper tree levels represented by the pool
$P = Z(2^r - 1) + \sigma$	Persistent pool capacity per recursive tree
D_ℓ, W_ℓ	Payload width and complete record width in bits
$n = P + B$	Number of tagged records shuffled per tree and batch
λ	Garbling security parameter, 128 bits

All MB, GB, TB, and PB in traffic/state tables are decimal. GiB and MiB are binary units and are used where required by storage billing. Costs include both table probes unless a row is explicitly labeled "per table."

2 Dictionary and recursive ORAM layout

We use two cuckoo tables, each with $2N$ logical cells, together with an eight-entry application cuckoo stash. The total table load is $N/(4N) = 1/4$. The original dictionary occupies 16 GiB or 256 GiB before hash-table, ORAM, and encoding overhead. A data payload has 256 key bits, 256 value bits, and one occupancy bit:

$$D_0 = 513, \qquad h_0 = d + 1, \qquad W_\ell = D_\ell + 2h_\ell + 1. \tag{1}$$

The two h_ℓ fields are the logical record identifier and assigned leaf; the last bit is the ORAM validity flag. Dictionary occupancy and ORAM validity are distinct. Recursive position-map records hold F leaf labels:

$$h_{\ell+1} = h_\ell - f, \qquad D_{\ell+1} = Fh_\ell. \tag{2}$$

Map packing changes record width; it does not change the binary tree's branching factor.

At each tree, the top r levels are represented by an unordered pool. Below them, a path contains

$$L_\ell = h_\ell - r + 1 \tag{3}$$

bucket levels. Public old-leaf paths determine which lower buckets are fetched. An ORAM logical identifier, an assigned leaf, and a physical slot are different objects. Revealing a requested old leaf does not reveal the record's identifier or its position within a bucket or pool.

Recursion stops at a configurable base cutoff or before the next tree would have fewer than $2B$ leaves. The final scalar position map has $C = 2^{h_{\text{last}}}$ entries of width h_{last}; its *full* capacity, not the requested cutoff, is charged. The selected layouts and all recursive record widths are listed in Section 7.

The overall structure follows Circuit OPRAM [1]. The concrete Boolean compilation below is a source-inspired planning template. In particular, $Z = 3$ and $\sigma = 128$ are fixed study parameters, not a proven overflow bound for these database sizes and access horizons.

3 Cryptographic primitives and representation costs

3.1 Hashing a secret-shared 32-byte query key

The query key is shared between the garbler and evaluator: $x = x_G \oplus x_E \in \{0,1\}^{256}$. Neither party is assumed to know x. Consequently, computing its two table addresses locally is not an admissible optimization. A concrete baseline evaluates AES-128-CMAC inside the secure computation, with a fixed, separately sampled secret PRF key K shared between the parties. The database uses the same function when it is constructed. For the fixed-length input $x = M_1 \parallel M_2$, CMAC is

$$\begin{aligned} L &= \mathrm{AES}_K(0^{128}), & K_1 &= \mathrm{dbl}(L), \\ C_1 &= \mathrm{AES}_K(M_1), & T &= \mathrm{AES}_K(C_1 \oplus M_2 \oplus K_1). \end{aligned} \tag{4}$$

Both message blocks are complete, so CMAC uses K_1, without a padding block. Multiplication by X in the binary field, denoted dbl, consists of a shift and XORs with selected bits of the constant reduction polynomial; it requires no AND gates. These choices follow AES and the CMAC specification [8, 9].

For an auditable gate count, use the established 32-AND AES S-box circuit listed by NIST, rather than assuming a newer optimized implementation. The ten AES rounds contain 160 S-boxes, giving $160 \cdot 32 = 5120$ ANDs for the encryption datapath. The AES-128 key schedule contains another 40 S-boxes, or 1280 ANDs. NIST lists 6400 ANDs for AES-128 with its key schedule and 5120 for the fixed-key datapath [10]. A secret, already expanded key has the same nonlinear datapath count: AddRoundKey remains XOR. Reusing the expanded key and K_1 across the B queries of a single batch therefore gives

$$A_{\text{hash}}(B) = \underbrace{1280 + 5120}_{6400 \text{ setup}} + \underbrace{2 \cdot 5120B}_{10240B \text{ query work}}. \tag{5}$$

At 32 bytes per half-gate AND, this is 200 KiB of setup plus 320 KiB per query. The primary model retains the 11 expanded round keys and K_1 as 1536 ordinary encoded state bits, using the same linked-garbling interface as other persistent state. Setup is therefore performed once per initialized state and amortized, rather than repeated in every batch. This requires compatible labels and distinct gate/hash tweaks across the composed computation; it does not permit reuse of one-time garbled tables. A reset to an incompatible encoding must pay for secure key-state translation or new input and setup. Equation (5) counts neither initial database hashing nor interface traffic.

One 128-bit PRF result suffices for two h-bit table addresses when $2h \le 128$: take two disjoint h-bit slices of T. For a pseudorandom T, the slices are jointly pseudorandom; a second CMAC is unnecessary for this address-generation interface. Equal keys deliberately receive equal address pairs. The full query key must still be compared with candidate keys to handle address collisions. This construction assumes

fixed-length, domain-separated use of the PRF; changing its application domain must preserve that separation.

Query-share input and result-share output belong in a separate interface ledger. Reconstructing $x_G \oplus x_E$ inside GC costs XORs but not ANDs. The model charges 4096 bytes for G's share labels and 12,288 bytes for E's input OT, giving 16,384 bytes per query: 12,288 forward and 4096 reverse, before framing. This is a convention, not an unavoidable lower bound: existing garbled inputs and correlated-OT encodings can change the charge. The value and hit flag remain shared; taking the two parties' label color bits gives local XOR shares as explained in Section 4. There is no plaintext answer decoding to E. Client delivery is a separate boundary and is not included in the inter-server totals.

3.2 Packed storage and conditional movement are separate costs

The proposed packed representation carries a 32-byte value in two 128-bit objects. We retain this as a payload-storage and transport optimization. It does not, by itself, specify the protocol for moving those objects under a secret control bit. In ordinary Boolean GC, a 128-bit label represents one Boolean wire; storing a payload in two opaque carriers is a different representation with its own input, output, and movement interfaces.

For this reason the primary total uses a fully specified Boolean movement cost: swapping a 256-bit payload under one secret bit costs 256 ANDs, with metadata added at its actual Boolean width. A conditional packed-backend column may instead use a measured or proved parameter c_2, the communication for swapping the two packed payload carriers under a secret control. Storage size alone does not establish c_2, and the conditional column must also include conversions and relabeling. When a shuffle-and-reveal step has made the physical selection public, both parties can select their own shares locally; that public routing genuinely avoids secret-control payload gates. This distinction makes the Boolean estimate auditable while keeping the packed optimization visible as a concrete implementation target.

3.3 One-hot garbling: what is obtained and what is charged

Let $P = 2^k$ be the padded size of a small array. Heath's binary-to-one-hot primitive creates $H(y)$ using $(k-1)\lambda$ bits of garbled material, or $16(k-1)$ bytes at $\lambda = 128$, when the evaluator knows y. The complete framework is not restricted to public true inputs: it masks private inputs and cancels the masks within secure modules [7, 6].

In particular, Section 5.1 of [6] encodes a private short integer $x \in \mathbb{Z}_{2^k}$ using a mask α and a one-hot encoding of $y = x + \alpha \bmod 2^k$. G knows α and E learns y, while neither learns x. For binary input, the stated construction adds α with a $(k-1)$-AND ripple-carry adder and then invokes binary-to-one-hot. With the paper's two-ciphertext Boolean AND implementation, the conversion material is

$$C_{\text{bin}\to\text{masked hot}} = 32(k-1) + 16(k-1) = 48(k-1) \text{ bytes.} \tag{6}$$

This excludes input-label transfer, the small reveal/interface bits, and any additional conversion required by the consuming operation. Binary and arithmetic one-hot outputs must not silently be identified: an arithmetic consumer may require the corresponding upcast or word conversion. The expanded vector still has P entries, so local work and working storage grow linearly in P for the binary construction, with further width factors for arithmetic words or payload outer products.

Equation (6) is a cost for a *masked-position* representation, not a canonical $H(x)$ at the same interface. A lookup into a mutable secret array must also align that representation with the array and account for any masking correction. The free truth-table evaluation $T(f)^{\mathsf{T}}H(y)$ uses a public constant table; it is not automatically a lookup into independently encoded secret array entries. Similarly, the cheap one-hot

outer product distributes one scalar or vector to one selected slot, not a different unknown payload at every slot. These observations do not rule out a useful module, especially with interaction and shuffled arrays; they identify the missing interface that must be specified. The primary totals therefore claim no generic one-hot discount for dynamic pool accesses.

3.4 One-AND swaps and a fused flat-array exchange

For a control bit s and two w-bit words a, b, define

$$d = s \wedge (a \oplus b), \qquad a' = a \oplus d, \qquad b' = b \oplus d. \tag{7}$$

The AND is bitwise with the same control on all bits. Thus a swap costs w ANDs and $3w$ XORs, rather than two independently implemented muxes. For P flat entries and a valid unique target index, start a carry word with the new value and conditionally swap it with each array entry at its one-hot selection bit. The final carry is the old value and the array contains the new one. A straightforward shared-prefix binary decoder uses $P - 2$ ANDs for $P \geq 2$, giving the specified circuit cost

$$A_{\text{flat exchange}}(P, w) = Pw + P - 2, \qquad X_{\text{flat exchange}}(P, w) = 3Pw + P - 2. \tag{8}$$

These are counts for this explicit decoder, not a claim of minimum circuit size. A one-entry array needs no selector circuit. An additional secret enable, out-of-range handling, or metadata predicates must be charged if present. The fused exchange assumes the replacement word is already available; an arbitrary function of the returned old word needs its own evaluation and suitable writeback. For batched repeated indices, a single unordered aggregate of selectors is insufficient: preserve sequential semantics or securely group duplicates and define their read/write order.

4 Interactive pool retrieval: protocol and security conditions

This section specifies a replacement for *pool retrieval*, conditional on the surrounding Circuit OPRAM modules maintaining their existing invariants. It is not a new proof of the complete ORAM construction. The replacement uses two-party secret-shared shuffling [2, 3]: the parties obtain fresh XOR shares of a common random permutation of the records, while neither party learns the composed permutation. Both parties remain semi-honest, and the application key and intermediate records remain secret shared. The ORAM architecture, recursive request processing, and eviction obligations remain those of the surrounding construction [1].

4.1 Interface and invariants

Fix one recursive level and one batch. Let P be the public pool capacity, B the public batch size, and u the number of UID bits at this level. A pool record contains a validity bit, UID, current leaf label, and payload. The pool is a *multiset stored in an array*; physical positions carry no meaning needed by subsequent pool processing. The following are required.

1. At most one valid record in the union of the pool and the lower tree has a given UID. Invalid records may contain arbitrary stale fields. When overlapping public paths are fetched, repeated appearances of one physical tree slot are aliases, not additional independent live records.
2. Existing private conflict processing supplies a representative bit r_j for each request $j \in \{1, \ldots, B\}$. There is at most one representative per UID, and representatives have distinct UIDs. Duplicate group membership and result propagation remain secret.

3. Each requested live block is obtained from exactly one of the pool and its fetched lower path. Invalid and nonrepresentative requests are masked. The surrounding algorithm privately combines these possibilities and clears each actual source once.
4. Later eviction and cleanup consume records through their secret metadata, rather than interpreting a pool position as a fixed tree node. If an implementation instead assigns persistent semantics to pool slots, it must first change that interface or pay for inverse routing.

These are interface conditions to verify in an implementation, not additional leakage permissions. In particular, the parties must not disclose whether a request hit the pool, the lower tree, or a dummy.

4.2 Match, tag, shuffle, and extract

The tag alphabet is $\{0, 1, \ldots, B\}$, with zero denoting an unselected row. A tag has $t = \lceil \log_2(B+1) \rceil$ bits. Tags identify public request *slots* in the current processing order; they do not disclose UIDs or secret duplicate groups. The following procedure is executed separately at each recursive level, with fresh shuffling randomness.

> **Input:** P secret records x_i, and B secret requests (q_j, r_j) in the surrounding circuit's current order.
>
> **1. Private matching.** For every i, j, compute
>
> $$e_{ij} = r_j \wedge \mathrm{valid}(x_i) \wedge [\mathrm{UID}(x_i) = q_j], \qquad h_j = \bigoplus_{i=1}^{P} e_{ij}.$$
>
> Set the tag of original row i to the bitwise XOR of the public binary constants j for which $e_{ij} = 1$. There is at most one such j.
>
> **2. Dummy completion.** Append one invalid record d_j per request. Give d_j tag j if $h_j = 0$, and tag zero otherwise. Its other fields have the same fixed width and sharing representation as an ordinary record. Nonrepresentatives have $h_j = 0$ and therefore receive their own dummy.
>
> **3. Secret shuffle.** Jointly shuffle the $n = P + B$ complete (tag, record) rows. Each party samples a private uniform permutation; the protocol applies their composition and refreshes shares. Use a fresh protocol instance and fresh correlations for this invocation.
>
> **4. Open tags only.** Reconstruct the shuffled tag column. For each j, both parties locally copy their record share at the unique row tagged j to output slot j. Neither party reconstructs the record or its validity bit. Output shares remain secret.
>
> **5. Retain the complement.** Both parties remove all B nonzero-tag rows locally and discard the public tags. Keep the remaining P records in their current unordered arrangement. Continue the existing private path/pool result selection, duplicate-result routing, recursive updates, and reinsertion/eviction schedule.

At every invocation, each nonzero tag occurs exactly once: either its real pool record or its dedicated dummy receives it. Consequently the number and pattern of opened tags do not reveal the number of hits. A single common dummy would be incorrect, as repeated opened positions would reveal misses or request equality. Tag assignment happens *before* shuffling, so no post-shuffle scan to rediscover dummy positions is needed.

Occupancy and clearing. Let H be the number of representative pool hits. Removal deletes H original valid records and $B - H$ appended dummies. The remaining H appended dummies replace the removed real records, leaving exactly P rows. This extracts and clears the pool sources in one operation. It is not necessary to privately compact the survivors or return them to their old slots. Appending the surrounding algorithm's B refreshed records and masked dummy returns gives the usual $P + B$

candidate bound. The existing additional eviction workspace, candidate selection, and cleanup are still required; this argument does not reduce their allowances.

Matching cost. Equality of two u-bit UIDs can be computed with $u-1$ ANDs, followed by two ANDs for validity and representative gating. Thus $BP(u+1)$ ANDs suffice for this matching kernel. The numerical ledger reserves one additional AND per pair, charging $BP(u+2)$ as a conservative interface allowance. Unique-hit aggregation is XOR. Multiplication of e_{ij} by a public tag bit is either copying or zero, so forming tags adds no payload-wide conditional selection. This count excludes the retained request-conflict module, path matching, and subsequent result selection. If valid-UID uniqueness is not established, XOR is not a substitute for a general OR or a first-match selection circuit.

4.3 Composition with recursion, paths, and updates

Private request sorting remains part of the protocol. The opened tag returns a result to the same *processing slot* that generated that tag. It does not reveal the private sorting permutation. Returning results to the original query order still requires the existing private inverse routing or an independently proved alternative. Public tag extraction cannot justify deleting those costs.

At a recursive level, several child requests may refer to one parent position-map block. Deduplication therefore uses that level's block UID, not merely the original application key. The parent block is fetched once; its different child offsets are read and updated privately, updates are coalesced, and one final parent block is committed. This shuffle does not make concurrent whole-block writes safe. For a read batch, duplicate queries can receive copies of the common result through existing secret routing. Supporting application writes additionally requires the chosen operation order to be implemented privately within each duplicate group.

The pool and lower-tree results must be combined using secret hit/validity flags. A no-pool-hit dummy is a placeholder, not evidence of an absent dictionary key. Likewise, selecting a real pool row does not authorize revealing that the lower path was unnecessary. The existing fixed path schedule is retained. Nonrepresentatives use the surrounding scheme's dummy-path rule, and collisions among public paths do not imply equality of logical UIDs. Shared physical slots on overlapping paths must have their extraction/clear masks aggregated before a single state update.

Fresh leaf labels and updated metadata stay secret. A removed valid pool record is reintroduced only through the existing refreshed-return module; an unselected record survives exactly once. No later step may both retain an old live copy and append a fresh live copy of the same UID. These conditions, together with correct coalescing of recursive updates, preserve the valid-UID uniqueness needed by the next invocation.

4.4 Security argument and its scope

Consider a hybrid in which matching, conflict processing, share conversion, and shuffling are replaced by their specified secure functionalities. Conditional on the prior transcript, the pre-shuffle tag vector contains exactly one copy of each $j \in \{1, \ldots, B\}$ and P zeros. A fresh uniform permutation maps it to a uniform ordered placement of the B distinct nonzero tags. This distribution is independent of the hit vector, the secret UIDs, and the original locations. A simulator can therefore sample the opened tag vector without learning any of those secrets.

One party knows its own private permutation, but the other party's independent uniform permutation still hides the composition. Secure shuffling must refresh the shares: simply permuting recognizable

active labels or unchanged share strings is not this functionality. Its output record shares reveal no values when tags are opened. Extraction and complement removal are deterministic local functions of the public tags and private shares, so they introduce no further openings. Subsequent secure modules consume these shares and reveal only outputs already authorized by the surrounding protocol. This is a conditional composition argument for the replacement module, not a security or overflow proof for the entire Circuit OPRAM template.

Every later batch must use a fresh hidden shuffle before disclosing further positions. Reusing a permutation permits cross-batch location linkage; reusing its masks or correlations can expose differences of secret arrays. Preprocessing can be performed ahead of time, but each correlation is consumed once and must not be restored after a retry or rollback. A retry uses a new correlation. In the semi-honest model, "discard" means that the protocol does not reuse a correlation; privacy must hold even against an adversary retaining its complete past view.

4.5 Representations and explicit communication charges

For the ordinary Boolean free-XOR backend, a bit b is represented by a garbler-held zero label K^0 and an evaluator-held active label $K^b = K^0 \oplus b\Delta$, where the permutation/color bit of Δ is one. Taking the color bits locally gives

$$[b]_G = \mathrm{col}(K^0), \qquad [b]_E = \mathrm{col}(K^b), \qquad [b]_G \oplus [b]_E = b.$$

This adds no communication and discloses nothing outside each party's existing view. It does not itself promise independent fresh masks across all output bits; the secure shuffle supplies the required refresh. No active label is sent to the garbler and no zero label is sent to the evaluator during this conversion.

Returning an XOR-shared bit to an ordinary GC can be charged conservatively as two party inputs followed by a free XOR: a 16-byte garbler input label, two 16-byte OT message corrections for the evaluator input, and a 16-byte per-OT matrix allowance for standard 128-bit IKNP extension. This gives *64 bytes per logical re-input bit*, plus amortized base-OT setup and transport padding. This is an explicit accounting choice rather than an optimal conversion claim. Re-input every retained or extracted field that the next GC actually consumes; the shuffle cost alone is not the complete retrieval cost. Persisting raw XOR shares instead of labels changes the storage backend and requires charging subsequent touched-state re-input.

The color-bit argument applies to one-label-per-Boolean-bit encodings. It does *not* turn two opaque packed labels into shares of a 256-bit value. Accordingly, an ordinary Boolean model plus the explicit conversion charge is the concrete baseline. A two-label payload scenario remains conditional on a specified packed encoding and secure conversion/update interface. One-hot and specialized garbling techniques may improve other small-array modules [6, 7], but they do not remove the representation or composition obligations here.

Shuffle cost. Let R include all actual XOR-share bytes in a row, including its tag. An n-input generalized Waksman network has

$$S(n) = n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$$

switches. In the construction of [3, Appendix A.3.1 and Figure 16], a switch uses a 1-out-of-2 OT with two $2R$-byte candidate payloads. Random-OT optimization reduces correction traffic from $4R$ to $2R$ bytes per switch. Wire masks are chosen in advance, so these OTs can be prepared in parallel; the data-dependent phase sends one masked input array per private permutation. Composing two private

permutations gives

$$C_{\text{shuffle}} = \underbrace{2nR}_{\text{online}} + \underbrace{4S(n)R}_{\text{switch corrections}} + C_{\text{OT}}.$$

Uniform permutations must be sampled first and then routed through the network; independent fair switch bits do not sample uniform permutations. All preprocessing communication belongs in the bandwidth/cost ledger even when it lies outside the batch's critical path. For $P = 317$, $B = 32$, and $n = 349$, $S(n) = 2630$. If a complete tagged row occupies 73 bytes, the first two terms are 50,954 and 767,960 bytes per batch. An illustrative 16-byte OT-extension allowance per switch adds 84,160 bytes across the two networks: 903,074 bytes per batch, or 28.22 decimal KB per request. If the 73-byte width excludes the tag, rounding to 74 bytes gives 28.57 KB per request instead. Opening the tag column to both parties adds at most $2\lceil nt/8 \rceil$ bytes with bit packing: 524 bytes per batch here, or 698 bytes if each tag is padded separately to one byte. Metadata matching, re-input, result routing, and the remaining ORAM work are separate charges.

5 Full batch execution and state dependencies

The two cuckoo tables execute in parallel, while each table's position-map chain is dependent. The logical schedule is:

1. Input the two shares of every query key; evaluate the secret-key PRF and retain both hash indices as encoded values.
2. Privately group duplicate logical requests at each recursive level. Do not reveal repeated old leaves directly. Representatives use their old positions; nonrepresentatives use independently random dummy paths.
3. Access and update the flat base map. Its returned old labels enable the next recursive tree to disclose its duplicate-masked old paths.
4. For that tree, scan lower-path metadata and perform the pool protocol. Combine the pool and path results under the one-live-copy invariant. Forward each representative result to duplicate requests privately, then restore request order using saved secret routing controls.
5. Coalesce updates to different offsets of the same recursive map record. Fetching one parent block does not authorize independent whole-block writes from its children. Commit a single combined replacement.
6. Insert refreshed records/dummies into the pool and execute the fixed eviction schedule: $2B$ lower subtrees per tree. Candidate selection and pool cleanup retain the template costs derived below.
7. Compare the full query key with both candidate keys and all eight cuckoo-stash keys. Return a shared value and shared hit bit.

Fresh leaf positions are generated as $\rho_G \oplus \rho_E$ inside GC, from independent private random contributions. Their XOR costs no AND gates; input labels and OTs are charged. Fresh positions are never opened when assigned. The old position is disclosed only when that record is accessed later. Untouched state labels persist; touched state obtains fresh output labels. Tables are generated for the current work and streamed, not retained as a complete multi-query garbled program.

Intersecting fetch paths require aggregation before clearing physical slots. After deduplication, at most one representative targets any live record. Per-slot deltas can therefore be XOR-aggregated and committed once. Deterministically chosen eviction subtrees are disjoint below the pool; this property does not make intersecting fetch writes independent.

6 Deriving the circuit and communication ledger

We count ordinary Boolean ANDs separately from OT and shuffle bytes. With FreeXOR and half gates, an AND contributes 32 bytes from G to E, while an XOR contributes no garbled table [5]. FreeXOR still requires local computation and memory traffic. This section makes explicit which counts follow directly from kernels and which are composition allowances.

6.1 Reusable Boolean kernels

For a bitonic sort of m items, let $K = 2^a$ with $a = \lceil \log_2 m \rceil$. There are

$$T(K) = Ka(a+1)/4 \tag{9}$$

compare-exchanges. For a k-bit sorting key, p payload bits, and v bits of inverse routing through saved controls, the template charges

$$A_{\text{sort}}(m, k, p, v) = T(K)(4k - 2 + p + v). \tag{10}$$

The comparison uses at most $3k - 2$ ANDs; swapping the key costs k, the payload costs p, and inverse routing costs v. Conditional swaps already use one AND per bit: no additional factor-of-two discount applies.

Two work-efficient segmented sweeps, adjacent group comparisons, and final masking are budgeted by

$$A_{\text{group}}(m, g, w) = (m-1)\max(g-1, 0) + 4(m-1)(w+2) + m(w+4). \tag{11}$$

The network and comparison formulas specify concrete kernels. Equation (11)'s use in complete pool/duplicate phases is a composition allowance; it is not a compiled gate trace. Define

$$\Phi(m, k, g, w, v) = A_{\text{sort}}(m, k, w+3, v) + A_{\text{group}}(m, g, w). \tag{12}$$

6.2 Work retained at each recursive tree

Suppress the level subscript and write h, W, L for the current widths and path length. The following counts are per table and per batch:

$$A_{\text{conflicts}} = A_{\text{sort}}(B, h + \log_2 B + 1, W + 2, W) + A_{\text{group}}(B, h, W), \tag{13}$$
$$A_{\text{pathfetch}} = BZL(h + W + 2), \tag{14}$$
$$A_{\text{depth}} = 2h(P + B), \tag{15}$$
$$A_{\text{select}} = \Phi(P + 3B, r + e + \lceil \log_2(P + 3B) \rceil + 2, r, W, W), \tag{16}$$
$$A_{\text{cleanup}} = \Phi(P + 3B, 1 + \lceil \log_2(P + 3B) \rceil, 1, W, 0), \tag{17}$$
$$A_{\text{pathchoice}} = Bh, \tag{18}$$

where $e = \max(1, \lceil \log_2(h+1) \rceil)$. Request-conflict work is scheduled on both sides of retrieval: forward grouping first, result propagation and inverse routing afterward. The old pool-retrieval sort is replaced in its entirety by the shuffle protocol, rather than discounted a second time.

The lower-subtree eviction budget is

$$\begin{aligned} A_{\text{evict}} = {} & (2BZL)(2h) + 2BL(Z-1)(3e - 2 + e + \lceil \log_2 Z \rceil + 1) \\ & + 2B \cdot 8L(6e+2) + 2B(2Z+4)LW. \end{aligned} \tag{19}$$

These terms are eligibility, bucket-winner selection, a parallel planner, and record movement. The planner factor 8 and extra movement factor 4 are exposed allowances inherited from the existing model. They must be checked against an implemented parallel eviction circuit; Equation (19) is not asserted as an exact count from the Circuit OPRAM paper.

6.3 Recursive updates, flat map, and application logic

For a map block containing F labels of h bits, write $U = Fh$ and $h' = h - \log_2 F$. Coalescing updates is budgeted by

$$A_{\text{packedmap}} = B\big((F-2) + F + 6U\big) + 4(B-1)(U+F+2) + (B-1)\max(h'-1, 0). \tag{20}$$

Here "packed" means multiple scalar position labels in one map block; it is unrelated to the two-label value encoding.

For the final flat map of C cells with h-bit labels, the fused read/update has cost

$$\begin{aligned} A_{\text{base}} = {} & A_{\text{sort}}(B, \log_2 C + \log_2 B + 1, h+2, h) + A_{\text{group}}(B, \log_2 C, h) \\ & + B\big((C-2) + C + Ch\big). \end{aligned} \tag{21}$$

The last line pays for decoding, representative gating, and one shared read/write product per label bit. It saves BCh ANDs per table relative to separate read and write masks. No unproved one-hot replacement is credited.

With two table candidates and eight cuckoo-stash entries, the application lookup contributes

$$A_{\text{app}} = B\big(10(255 + 1 + 256) + 9\big) = 5129B. \tag{22}$$

The 255 ANDs test a 256-bit equality, one gates occupancy, 256 mask the returned value, and nine combine hit flags. A dictionary key is assumed unique among live entries.

6.4 The new pool term and directional traffic

Let $t = \lceil \log_2(B+1) \rceil$ and $b_\ell = \lceil (W_\ell + t)/8 \rceil$. For each pool and batch, the model charges

$$A_{\text{poolmatch}} = BP(h_\ell + 2), \tag{23}$$

$$C_{\text{shuffle}} = 2nb_\ell + 4S(n)b_\ell + 32S(n), \tag{24}$$

$$C_{\text{tags}} = 2\lceil nt/8 \rceil, \tag{25}$$

$$I_{\text{reinput}} = nW_\ell \quad \text{shared bits.} \tag{26}$$

The $32S(n)$ term is an illustrative 16-byte OT-extension allowance for each of the $2S(n)$ switches. Actual OT setup, padding, and implementation constants must be measured. Shuffle corrections are paid for every fresh batch, even when prepared offline. The tag vectors are bit-packed; rounding is per message.

The primary model converts all shuffled records back to Boolean labels, including the selected records and remaining pool. A shared-bit input uses 16 bytes for the garbler's share label and classic evaluator input OT (32 forward, 16 reverse): 48 bytes forward and 16 reverse. This is a conservative standard interface, not an optimal mixed-protocol conversion. Tag bits are opened and do not need re-input. The shuffle and tag traffic is symmetric across the two directions because the two private permutations exchange the parties' roles.

Let A be total ordinary ANDs per *complete query*, including both tables, application logic, and 10,240 CMAC ANDs; let J be the corresponding shared re-input bits, $U = 2\sum_\ell h_\ell$ the fresh-position bits, $V = U$ the old-path bits, and C_s the sum of shuffle and tag bytes per query. Before framing,

$$C_{G\to E} = 32A + 48J + \tfrac{1}{2}C_s + 48(256) + 48U + V/8, \tag{27}$$

$$C_{E\to G} = 16J + \tfrac{1}{2}C_s + 16(256) + 16U + V/8. \tag{28}$$

Old paths are opened to both parties, costing $V/8$ bytes in each direction. The query key is entered as two 256-bit shares. No supplied-index inputs are charged, because indices are computed internally. No plaintext answer decoding to E is charged: the answer remains shared. Base OT, secret PRF key setup, initialization, and authorized client delivery are separate amortized or deployment-specific items. The reported wire counts multiply each direction by 1.05 for framing.

7 Numerical results and selection of parameters

7.1 Batch and recursion search

The search enumerates $B \in \{8, 16, 32, 64, 128, 256\}$, $F \in \{4, 8, 16, 32\}$, and requested base cutoffs $\{16, 32, 64, 128, 256, 512, 1024, 2048, 4096\}$: 216 combinations per database size. Each row below selects the cheapest network/state/I/O configuration at that batch size. Both final selections use $B = 32$, $F = 4$, and an actual base map of 2048 entries. The requested cutoff may have several equivalent values; the actual instantiated base is what matters. Small batches lose amortization of the fixed 128-record pool slack. Larger batches grow the remaining padded sorting networks. Consequently the objective is not monotone in batch size.

B	F_{28}/F_{32}	MB/q, 28	MB/q, 32	\$/yr, 28	\$/yr, 32	τ_B limit (ms)
8	4/4	342.7	429.9	\$55.89M	\$71.97M	0.56
16	4/4	395.4	494.2	\$64.30M	\$82.26M	1.12
32	4/4	237.8	301.3	\$39.05M	\$51.40M	2.24
64	4/8	273.3	342.2	\$44.70M	\$57.83M	4.48
128	8/8	314.9	392.5	\$51.30M	\$65.88M	8.96
256	8/8	366.0	454.3	\$59.37M	\$75.60M	17.92

Traffic includes both directions and 5% framing; prices exclude CPU sizing. The final column is the one-state batch reuse limit, not a predicted service time. Padding effects and the fixed slack make this an optimum of the selected implementation template, not a lower bound for OPRAM.

7.2 Instantiated recursive trees

Every selected tree has a 317-record pool, $r = 6$, and a shuffle of 349 records. The final map has 2048 scalar entries of 11 bits. The table lists each tree's address width h, payload width D, complete record width W, and lower-path length $L = h - 5$.

Level	h_{28}	D_{28}	W_{28}	L_{28}	h_{32}	D_{32}	W_{32}	L_{32}
0	29	513	572	24	33	513	580	28
1	27	116	171	22	31	132	195	26
2	25	108	159	20	29	124	183	24
3	23	100	147	18	27	116	171	22
4	21	92	135	16	25	108	159	20
5	19	84	123	14	23	100	147	18
6	17	76	111	12	21	92	135	16
7	15	68	99	10	19	84	123	14
8	13	60	87	8	17	76	111	12
9	11	52	75	6	15	68	99	10
10	–	–	–	–	13	60	87	8
11	–	–	–	–	11	52	75	6

There are 10/12 dependent path-disclosure stages. Choosing fanout 8 for the larger database offers a latency/storage tradeoff; the selected fanout 4 minimizes the stated aggregate price objective, which does not price round-trip latency.

7.3 Ordinary garbled-circuit work

The following rows are millions of ANDs per complete query, already summed over both tables and every recursive level. They exclude OT and shuffle messages, which are accounted separately.

Component (million AND/query)	2^{28}	2^{32}
Request grouping and private result routing	0.082943	0.103014
Lower-path retrieval and clearing	0.205776	0.288648
Pool UID matching and tags	0.139480	0.182592
Eviction-candidate depth	0.008725	0.011517
Pool eviction selection	3.270576	4.008054
Pool cleanup	1.723449	2.110749
Lower-subtree eviction	1.479984	2.071104
Duplicate-masked path selection	0.000400	0.000528
Coalesced recursive-map updates	0.015771	0.021057
Fused flat position-map exchange	0.054744	0.054744
Dictionary key match and value selection	0.005129	0.005129
AES-CMAC query frontend	0.010240	0.010240
Total	**6.997218**	**8.867375**

For $N = 2^{28}$, eviction selection and pool cleanup account for 71.4% of ordinary ANDs.

For $N = 2^{32}$, eviction selection and pool cleanup account for 69.0% of ordinary ANDs.

These retained phases dominate after retrieval has been improved. Optimizing the shuffle alone cannot remove them.

7.4 Pool protocol: all costs, not only online bytes

The main pool terms below include both tables and all trees, per query. Re-input is charged even for the data already moved by the shuffle; these are distinct operations. The unframed total is comparable to the old pool phase only, not to the entire query.

Pool component (MB/query)	2^{28}	2^{32}
Metadata match tables	4.463360	5.842944
Online masked arrays	0.009772	0.011997
Offline switch corrections	0.147280	0.180812
OT-extension allowance	0.052600	0.063120
Tag-share opening	0.000327	0.000393
GC re-input, both directions	2.343884	2.882740
Total pool retrieval	7.017224	8.982006

7.5 Complete directional communication

Traffic component (MB/query)	2^{28}	2^{32}
Garbled tables, forward	223.910968	283.756016
Pool re-input, forward	1.757913	2.162055
Pool re-input, reverse	0.585971	0.720685
Shuffle + tags, both directions	0.209980	0.256322
Query shares, both directions	0.016384	0.016384
Fresh-position inputs, both directions	0.025600	0.033792
Old-path disclosure, both directions	0.000100	0.000132
Forward, after framing	237.095679	300.388127
Reverse, after framing	0.736582	0.904529
Total, after framing	**237.832261**	**301.292656**

PRF setup is 6400 ANDs plus 8192 input bytes per initialized state, before framing, and is not amortized into these per-query rows. At an epoch of T queries, add $(32 \cdot 6400 + 8192)/T$ bytes per query before framing, together with any state handoff. Base OT and initial database construction remain separate.

7.6 Packed-value sensitivity without an unsupported gate discount

A 256-bit ordinary Boolean value uses 4096 label bytes per party. A two-carrier representation would use 32 bytes. For M_{value} physical data/stash slots, the conditional state saving is exactly $4064M_{\text{value}}$ bytes per party. Recursive leaf labels are unchanged.

State per party (TB)	2^{28}	2^{32}
Primary Boolean representation	64.699	1,061.578
Conditional two-carrier values	38.517	642.665

This is a storage sensitivity, not a complete second deployment estimate. A packed backend must specify how shared values enter and leave its encoding and what a secret-controlled move costs. The present shuffle transports ordinary XOR shares; the Boolean color-bit conversion does not prove a conversion for two opaque carriers. The earlier hypothetical "two half-gate equivalents per value move" cannot be included without those interfaces. Ordinary compact XOR-share storage is another valid design direction, but needs re-input accounting on every subsequently touched field.

8 Persistent state, I/O, and annual capacity cost

The number of persistent slots in a tree is

$$Z\left(2^{h_\ell+1} - 2^r\right) + P. \tag{29}$$

Therefore the primary model's persistent logical bits are

$$S_{\text{bits}} = 2\left[\sum_{\ell}\left\{Z(2^{h_\ell+1}-2^r)+P\right\}W_\ell + Ch_{\text{last}}\right] + 8\cdot 513 + 1536. \tag{30}$$

Each party retains one 16-byte label per logical bit, so the total across both parties is $32S_{\text{bits}}$ bytes. The garbler needs one zero label plus a global FreeXOR offset, not two independently stored labels per bit. The final 1,536 bits retain the expanded AES round keys and CMAC subkey. The newly added dummy rows and shuffle masks are batch workspace, not persistent additional tree capacity.

The persistent-I/O envelope touches the whole pool, B fetch paths, and $2B$ eviction paths at every tree. It counts overlaps repeatedly and assumes scratch intermediates remain in RAM:

$$T_{\text{bits}} = 2\left[\sum_{\ell}(P+3BZL_\ell)W_\ell + Ch_{\text{last}}\right] + 8\cdot 513. \tag{31}$$

Reading and writing label state costs $32T_{\text{bits}}/B$ bytes per query *per party*. Converting pool labels to XOR shares does not imply that the rest of the database has been converted to compact share storage.

8.1 Historical deployment assumptions

The dollar estimates reuse the repository's official AWS US East (N. Virginia) price snapshot, published 12 October 2026. They are not a new price quote. The assumed c6in.32xlarge rate is \$7.2576/hour; each node has 256 GiB RAM, 200 Gbit/s network, and up to 100 Gbit/s EBS bandwidth. The gp3 capacity rate is \$0.96/GiB-year [11, 12, 13].

Applying network-efficiency, distribution, and utilization factors gives

$$c_{\text{NIC}} = \frac{200\cdot 10^9}{8}(0.65)(0.70)(0.70) = 7.9625\cdot 10^9 \text{ bytes/s per direction.} \tag{32}$$

Full-duplex NIC capacity is enforced at both roles:

$$n_{\text{role}} = \left\lceil \frac{Q\max(C_{G\to E}, C_{E\to G})}{c_{\text{NIC}}} \right\rceil, \qquad D_{\text{EC2}} = 2n_{\text{role}}(8760)(7.2576). \tag{33}$$

Both servers' worker fleets are in one availability zone; the model assigns no marginal cross-AZ transfer charge in this placement. Intra-party data distribution is not free in an implementation and is not explicitly routed by this aggregate NIC calculation.

For storage, annual capacity cost is $0.96\,S_{\text{bytes}}/2^{30}$. The inherited I/O scenario uses 64 KiB aggregated operations, provisioned with $0.65\cdot 0.70$ throughput headroom, gp3 limits of 2,000 MiB/s and 80,000 IOPS per volume, and at least four IOPS per provisioned MiB/s. It prices extra throughput at \$0.48/(MiB/s)-year and extra IOPS at \$0.06/IOPS-year after each volume's included 125 MiB/s and 3,000 IOPS. This is a specified provisioning scenario, not a minimal price or a disk latency guarantee. The model also checks volume attachments and EBS host capacity, taking a maximum with the NIC fleet rather than double-counting the same host.

8.2 Selected annual budgets

Both-party cost unless stated	2^{28}	2^{32}
NIC hosts per party	298	378
Additional capacity/I/O EC2/year	\$0.00M	\$0.00M
Network-sized EC2/year	\$37.89M	\$48.06M
SSD capacity/year	\$0.12M	\$1.90M
Modeled persistent I/O/year	\$1.04M	\$1.43M
Combined annual capacity budget	**\$39.05M**	**\$51.40M**
Persistent R+W/query/party (MB)	18.777	25.919
State capacity, both parties (TB)	129.399	2,123.157

The minimum streaming buffer allowance is 64 MiB per party. It is not the total RAM requirement: workers, in-flight circuit labels, OT matrices, shuffle correlations, network buffers, and eviction scratch must be provisioned. No full-horizon garbled program is retained.

8.3 Known-work CPU sensitivity

At the following assumed effective ordinary-AND rates per node, take the maximum of NIC, storage/I/O, and ordinary-gate host counts. These figures include the stated state and I/O budgets, but *exclude* the additional CPU cost of OT, shuffle, free-XOR, serialization, and distributed execution. They therefore are not complete CPU-priced service budgets.

Million AND/s/node	Hosts/party, 28	Hosts/party, 32	\$/yr, 28	\$/yr, 32
50	1400	1774	\$179.17M	\$228.90M
200	350	444	\$45.66M	\$59.79M
1,000	298	378	\$39.05M	\$51.40M

9 Throughput, interaction, and unresolved costs

Ordinary Yao evaluation needs no network round trip per AND layer. However, old-path discovery, the two online private permutations, opening tags, and re-entering shared state create real dependencies. The number of recursive trees is only the count of dependent *path-disclosure* barriers, not the protocol's total interaction rounds. The two tables can proceed in parallel; a concrete implementation must schedule the additional interfaces and account for their RTTs.

Let τ_B be the measured time until one independently evolving ORAM state can service its next batch, including preprocessing availability, evaluation, updates, and synchronization. With K independent states and target utilization $u = 0.70$, a necessary capacity condition is

$$Q \le \frac{uKB}{\tau_B}, \qquad K \ge \left\lceil \frac{Q\tau_B}{uB} \right\rceil. \tag{34}$$

At $B = 32$ and $Q = 10{,}000$, one state requires $\tau_B \le 2.24$ ms. Mean batch-fill delay is approximately $(B-1)/(2Q) = 1.55$ ms under regular aggregate arrivals. Neither bound is established by buying enough network nodes. A large distributed fleet does not automatically parallelize a single dependent state transition.

Independent replicas multiply state capacity and initialization work. They need independent hidden ORAM layouts; replicating the underlying read-only dictionary does not authorize reusing correlated

randomness or garbled tables. Application writes would additionally require consistency across replicas. The one-state annual figures must be revised if measured τ_B requires additional states.

CPU accounting. An ordinary-AND rate can provide a workload sensitivity, but it does not account automatically for OT extension, Waksman routing, PRG expansion, label-to-share conversions, FreeXOR memory traffic, or distributed workers. The report's rate sensitivities price known ordinary-gate work only; the extra kernels still require benchmarking. Offline communication and CPU must sustain the same long-run query rate as online evaluation.

What remains to be established. The match–tag–shuffle argument is conditional on the underlying OT, garbling, and shuffle security, and on the unique-live-copy and duplicate handling invariants. It is not a stand-alone proof for an implemented Circuit OPRAM. Required next validations are: a full recursive-state implementation; overflow and cuckoo-stash bounds at the desired lifetime; an exact parallel eviction and packed-map-update circuit; hidden initial placement; fresh-correlation and label-lifetime management; end-to-end batch service time; and failure recovery. These are material limits of the estimate, not priced deployment features.

10 Reproduction and implementation checkpoints

The accompanying Python model adds a new calculation rather than changing the historical comparison files. From the repository root:

```
python -m mpc_cost.interactive_shuffle_cost
python -m unittest mpc_cost.test_interactive_shuffle -v
python -m mpc_cost.render_interactive_shuffle_report
```

The JSON ledger is `mpc_cost/results/interactive_shuffle_comparison.json`. The source model is `mpc_cost/interactive_shuffle_cost.py`; its inherited kernel formulas are in `mpc_cost/two_party_batch.py`. The historical price inputs are in `mpc_cost/data/aws_garbled_prices_2026-10-13.json`. The PDF can be rebuilt from the self-contained delivered LaTeX source by running `pdflatex` twice. The source requires only standard packages listed in its preamble and has an embedded bibliography.

Functional checks should verify that tags always form the same fixed multiset, removal leaves exactly P rows, dummies hide misses and duplicates, and clear/reinsert operations preserve one live record per UID. Arithmetic checks should independently reconcile each direction of traffic, all pool terms, the flat-map saving, the PRF count, storage, and NIC rounding. These checks validate the accounting and small functional kernels; they are not cryptographic benchmarks or a security proof.

References

[1] T.-H. Hubert Chan and Elaine Shi. *Circuit OPRAM: Unifying Statistically and Computationally Secure ORAMs and OPRAMs*. IACR ePrint 2016/1084, revised 2020. https://eprint.iacr.org/2016/1084.

[2] Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. *Secret-Shared Shuffle*. ASIACRYPT 2020, ePrint 2019/1340. https://eprint.iacr.org/2019/1340.

[3] Gayathri Garimella, Payman Mohassel, Mike Rosulek, Saeed Sadeghian, and Jaspal Singh. *Private Set Operations from Oblivious Switching*. PKC 2021, ePrint 2021/243, Appendix A.3. https://eprint.iacr.org/2021/243.

[4] Feng Han et al. *Concretely Efficient Correlated Oblivious Permutation*. IACR ePrint 2025/449. https://eprint.iacr.org/2025/449.

[5] Samee Zahur, Mike Rosulek, and David Evans. *Two Halves Make a Whole: Reducing Data Transfer in Garbled Circuits Using Half Gates*. EUROCRYPT 2015. https://www.cs.virginia.edu/~evans/pubs/ec2015/.

[6] David Heath. *Efficient Arithmetic in Garbled Circuits*. EUROCRYPT 2024, ePrint 2024/139, Sections 4–5. https://eprint.iacr.org/2024/139.

[7] David Heath and Vladimir Kolesnikov. *One Hot Garbling*. CCS 2021, ePrint 2022/798, Sections 4–5. https://eprint.iacr.org/2022/798.

[8] NIST. *Advanced Encryption Standard (AES)*. FIPS 197, updated 2023. https://doi.org/10.6028/NIST.FIPS.197-upd1.

[9] NIST. *Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication*. SP 800-38B, updated 2016. https://doi.org/10.6028/NIST.SP.800-38B.

[10] NIST. *Circuit Complexity: List of Circuits*. Accessed 16 October 2026. The primary budget uses the classic 32-AND AES S-box, not the newer 28-AND variants. https://csrc.nist.gov/projects/circuit-complexity/list-of-circuits.

[11] AWS. *Amazon EC2 C6i/C6in Instances*; *EC2 Instance Network Bandwidth*. https://aws.amazon.com/ec2/instance-types/c6i/; https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html.

[12] AWS. *Amazon EBS Pricing*; *General Purpose SSD Volumes*. https://aws.amazon.com/ebs/pricing/; https://docs.aws.amazon.com/ebs/latest/userguide/general-purpose.html.

[13] AWS. *AmazonEC2 US East (N. Virginia) Price List*, version 20261012193925, published 12 October 2026; locally retained snapshot. https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/20261012193925/us-east-1/index.csv.